

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина: «Информационные системы»

Курсовая работа. Часть 1.

Выполнили: Рогович Мария Р3313

Леонтьева Арина Р3313

Преподаватель: Кобик Никита Алексеевич

г. Санкт-Петербург 2026 год

Содержание:

<i>Задание</i>	<i>3</i>
<i>Текстовое описание предметной области</i>	<i>4</i>
<i>Решаемые задачи.....</i>	<i>6</i>
<i>Функциональные/нефункциональные требования.....</i>	<i>7</i>
<i>Основные прецеденты</i>	<i>9</i>
<i>Архитектура.....</i>	<i>12</i>

Задание

1. Согласовать с преподавателем предметную область, для которой будет разрабатываться информационная система.
2. Составить подробное текстовое описание предметной области.
3. Сформулировать, зачем нужна информационная система для представленной предметной области, какие задачи она позволит решить.
4. Составить функциональные/нефункциональные требования к разрабатываемой информационной системе.
5. Построить модели основных прецедентов (прецеденты согласуются с преподавателем), составить их описание.
6. Предложить архитектуру будущей системы. При составлении архитектуры необходимо учитывать, что все этапы курсовой работы необходимо будет демонстрировать на сервере **helios**. Согласовать с преподавателем технологии и фреймворки, которые будут использоваться при реализации системы. Для реализации системы можно использовать:
 - a. Frontend: React, Angular, Vue, Next JS, JSF, Spring MVC (Thymeleaf или другой шаблонизатор).
 - b. Backend: основанный на Jakarta EE или Spring MVC
 - c. БД: PostgreSQL
7. Составить отчет.

Текстовое описание предметной области

Предметной областью информационной системы является автоматизация проведения парусных гонок с пересадками. В таких гонках количество экипажей превышает количество доступных яхт. Чтобы все экипажи могли выйти на воду, соревнования проводятся сериями гонок, между которыми экипажи меняются лодками, а часть экипажей ожидают следующей гонки на суше.

Ключевые аспекты предметной области:

1. Пользователи системы:

- Организатор – член гоночного комитета, который составляет расписание пересадок.

2. Лица, участвующие в соревновании:

- Спортсмен – лицо, которое участвует в гонке или намеревается участвовать в гонке на соревновании.
- Экипаж – группа спортсменов, заявленная на соревнование.

3. Материальная часть:

- Яхта – парусное судно, имеющее уникальный номер.

4. Составляющие части соревнований:

- Гонка – состязание яхт в скорости прохождения заранее определенной дистанции. Время гонки может меняться.
- Соревнование (регата) – серия гонок.
- Пересадка – смена яхты экипажем между гонками.

5. Формирование пересадочного листа:

- Ввод участников соревнований: название учебного заведения, фамилия рулевого.
- Ввод доступных яхт: их количество, порядковый номер, указанный на самой лодке.
- Ввод максимально допустимого количества гонок за все дни соревнования.
- Ввод количества дней соревнований.
- Ввод минимального количества гонок для одного экипажа за все дни соревнования.
- Выполнение алгоритма оптимизации по введенным данным.

- Формирование официального документа для гоночного комитета.

Решаемые задачи

Главная сложность парусных гонок с пересадками — это составление расписания. Оно должно одновременно удовлетворять нескольким требованиям: все команды проходят одинаковое количество гонок, каждая команда встречается с каждой другой командой одинаковое число раз, каждая команда проходит одинаковое число гонок на каждой яхте, число пересадок при этом минимально.

Сейчас такое расписание подбирается вручную.

1. Регистрация экипажей:

Организатор регистрирует экипаж по наименованию университета, фамилии рулевого.

2. Учёт яхт и фиксация исходных данных для расписания:

Система хранит список яхт, доступных для гонок. После завершения регистрации количество экипажей и яхт считается окончательным, и на основе этих данных строится расписание.

3. Автоматическое составление расписания гонок:

Система определяет, какие экипажи выходят на воду в какой гонке и на каких яхтах.

4. Учёт требований к расписанию:

Система стремится соблюсти сразу все перечисленные требования.

5. Формирование финального документа

Система формирует пересадочный лист.

Функциональные/нефункциональные требования

Функциональные требования

- FR1. Система должна позволять создать новую учетную запись, указав логин и пароль.
- FR2. Система должна позволять войти организатору, указав логин и пароль.
- FR3. Система должна отклонять регистрацию экипажа, если экипаж с такой информацией уже зарегистрирован в этом соревновании, и сообщать об этом организатору.
- FR4. Система должна позволять организатору изменять и удалять данные зарегистрированного экипажа.
- FR5. Система должна позволять организатору добавить яхту, указав ее уникальный номер.
- FR6. Система должна позволять организатору изменять и удалять номер яхты.
- FR7. Система должна позволять организатору задать максимальное количество гонок в регате перед формированием расписания.
- FR8. Система должна позволять организатору задать минимальное требуемое количество гонок для одного экипажа в рамках всех гоночных дней.
- FR9. Система должна после завершения регистрации формировать финальный документ.
- FR10. Система должна для каждой гонки указывать экипажи, выходящие на воду, и яхту, назначенную каждому из них, а остальные экипажи отмечать, как остающиеся на берегу.
- FR11. Система должна формировать расписание так, чтобы в каждой гонке на воду выходило число экипажей, равное числу яхт, каждая яхта была назначена ровно одному экипажу, а каждый экипаж был назначен не более чем на одну яхту.
- FR12. Система должна стремиться минимизировать разницу между максимальным и минимальным числом гонок среди всех экипажей.
- FR13. Система должна стремиться минимизировать разницу между максимальным и минимальным числом встреч среди всех пар экипажей.
- FR14. Система должна стремиться минимизировать для каждой яхты разницу между максимальным и минимальным числом гонок, проведенных на ней разными экипажами.
- FR15. Система должна стремиться минимизировать общее число пересадок.
- FR16. Система должна позволять организатору просматривать итоговое расписание и скачивать его.

Требования к удобству использования

- UR1. Система должна предоставлять пользовательский интерфейс на русском языке.
- UR2. Система должна позволять зарегистрировать экипаж заполнением одной формы без перехода на другие страницы.

- UR3. Система должна корректно отображаться и работать в браузерах Chrome, Firefox, Safari и Яндекс Браузер актуальных версий при разрешении экрана не менее 1280×720.

Требования к надёжности

- RR1. Система должна хранить данные об экипажах, яхтах в хранилище так, чтобы они сохранялись после перезапуска системы.
- RR2. Система должна при сбое в процессе формирования расписания оставлять ранее сохранённые данные без изменений и показывать организатору сообщение об ошибке.
- RR3. Система должна обеспечивать восстановление данных из резервной копии.

Требования к производительности

- PR1. Система должна формировать один вариант расписания не более чем за 5 секунд при максимальных размерах.
- PR2. Система должна выполнять остальные операции (регистрацию, изменение, просмотр расписания) не более 1 секунды.
- PR3. Система должна поддерживать одновременную работу не менее 3 организаторов.

Основные прецеденты

Прецедент: Регистрация в системе
ID: 1
Краткое описание: Регистрация организатора
Главный актер: Организатор (или пользователь)
Предусловия: -
Основной поток: 1. Организатор заходит на страницу системы. 2. Вводит электронную почту и пароль для создания учетной записи. 3. Система регистрирует пользователя.
Альтернативный поток: Организатор уже зарегистрирован в системе, тогда она выдает ошибку регистрации и предлагает обычный вход.
Постусловия: Пользователь заходит в систему

Прецедент: Вход в систему
ID: 2
Краткое описание: Вход организатора в систему
Главный актер: Организатор (или пользователь)
Предусловия: Пользователь уже зарегистрирован
Основной поток: 1. Организатор заходит на страницу системы. 2. Вводит электронную почту и пароль. 3. Система перенаправляет пользователя на главную страницу.
Альтернативный поток: Организатор ввёл неверные данные, тогда она выдает ошибку входа и предлагает ввести данные заново.
Постусловия: Пользователь заходит в систему

Прецедент: Ввод данных об экипаже
ID: 3
Краткое описание: Пользователь регистрирует экипаж
Главный актер: Организатор (или пользователь)
Предусловия: Пользователь находится на главной странице
Основной поток: 1. Система запрашивает ввод данных для экипажа 2. Пользователь вводит наименование университета и фамилию рулевого 3. Система предлагает ввод еще одного экипажа или перейти к вводу информации о яхтах

Прецедент: Ввод данных об экипаже
Альтернативный поток: Организатор ввёл данные уже добавленного экипажа, система выдает ошибку и предлагает изменить данные.
Постусловия: Пользователь переходит на страницу ввода информации о яхтах

Прецедент: Ввод данных о яхтах
ID: 4
Краткое описание: Пользователь вводит информацию о доступных яхтах
Главный актер: Организатор (или пользователь)
Предусловия: Пользователь находится на главной странице
Основной поток: 1. Система запрашивает ввод данных для яхты 2. Пользователь вводит уникальный номер яхты 3. Система предлагает ввод еще одной яхты или перейти на страницу ввода ограничительных условий
Альтернативный поток: Организатор ввёл данные уже добавленной яхты, система выдает ошибку и предлагает изменить данные.
Постусловия: Пользователь переходит на страницу ввода ограничительных условий

Прецедент: Ввод ограничительных условий
ID: 5
Краткое описание: Пользователь вводит информацию об ограничительных условиях
Главный актер: Организатор (или пользователь)
Предусловия: Пользователь зарегистрировал экипажи и яхты
Основной поток: 1. Система запрашивает ввод данных для яхты 2. Пользователь вводит максимальное допустимое количество гонок, минимальное количество гонок для одного экипажа, количество дней соревнований 3. Система предлагает сформировать финальное расписание
Альтернативный поток: Организатор ввёл неверные данные и хочет вернуться на предыдущие страницы для изменения информации.
Постусловия: Пользователь переходит на страницу с составленным расписанием

Прецедент: Просмотр составленного расписания
ID: 6
Краткое описание: Пользователь просматривает составленный гоночный лист
Главный актер: Организатор (или пользователь)

Прецедент: Просмотр составленного расписания
Предусловия: Пользователь зарегистрировал экипажи, яхты и ввел ограничительные условия.
Основной поток: Система выводит составленное расписание и предлагает скачать документ
Альтернативный поток: Организатор ввёл неверные данные и хочет вернуться на предыдущие страницы для изменения информации для составления нового расписания.
Постусловия: Пользователь скачивает гоночный лист.

Архитектура

Клиент-серверная архитектура

Система построена на клиент-серверной архитектуре: frontend и backend разделены и взаимодействуют через REST API.

Фронтенд

Реализован на Vue.js.

Бэкенд

Реализован как монолит на Spring Boot с классической слоистой архитектурой: Controller → Service → Repository.

Основные модули бэкенда:

Модуль авторизации — аутентификация и авторизация пользователей на основе JWT-токенов.

Модуль оптимизации расписания — сервисный слой, реализующий алгоритм построения расписания с учётом пересадок.

Модуль управления экипажами — регистрация экипажей, привязка к яхтам.

База данных

PostgreSQL — хранит данные о яхтах, экипажах (университет и фамилия рулевого), организаторах.

Развёртывание

Все серверные компоненты развёрнуты на сервере helios. Frontend взаимодействует с backend по протоколу HTTP через REST API.

